

Lucas Dame
Adrien Lefebvre
Vincent Guichard
Paul-Antoine Paris

Rapport **PROJET INDUSTRIE**

ANDRA 2025/2026

Tuteur ANDRA :
Guillaume Hermand
Tuteurs Ecole des Mines :
Antoine Richard
Romain Serizel

Avant-Propos et Remerciements

Ce rapport a été rédigé à la suite du projet industrie de l'École des Mines de Nancy, en partenariat avec l'ANDRA, durant l'année scolaire 2025-2026. Il a été réalisé par les auteurs avec l'aide des ingénieurs du TechLab, de nos tuteurs ainsi que de l'intervenant de l'ANDRA. Nous tenons à remercier chaleureusement l'ensemble des personnes qui ont pu nous aider et nous conseiller tout au long de ce projet. L'ensemble de notre projet se trouve à l'adresse GitHub suivante : https://github.com/Hexa-Da/ANDRA_2025_2026

Table des matières

Avant-Propos et Remerciements	1
1 Introduction	4
2 État de l'art et du projet	5
2.1 Détection de fissures	5
2.2 Positionnement et navigation	6
3 Semestre 1	6
3.1 Prise en main du robot	6
3.1.1 Présentation des parties indépendantes	6
3.1.2 Remise en service et configuration matérielle	8
3.1.3 Projet parallèle : Vision à 360 degrés	8
3.2 Reprise des travaux de l'année 2024-2025	9
3.3 Préparation de la première descente	9
3.3.1 Outillage et reproductibilité	10
3.3.2 État du robot avant la descente	10
3.3.3 Objectifs de la descente	10
3.4 Résultats de la première descente	11
3.4.1 Succès opérationnels	11
3.4.2 Incident matériel	12
3.4.3 Constats et axes d'amélioration	12
4 Semestre 2	12
4.1 Résolution du problème LiDAR	12
4.2 Optimisation YOLO sur Jetson	13
4.3 Travaux préparatoires avant la seconde descente	13
4.3.1 Révision de la séquence PTZ	13
4.3.2 Reconstruction de l'arbre TF	13
4.3.3 Amélioration de la qualité des scans LiDAR	14
4.3.4 Mise à jour EKF + SLAM	14
4.3.5 Sauvegarde de l'image disque	14
4.4 Résultats de la seconde descente	15
4.5 Navigation autonome sur carte	15
4.6 Fusion navigation + acquisition	16
4.6.1 Compromis mécanique LiDAR/PTZ	16
4.6.2 Orchestration logicielle	17
4.7 Amélioration du dataset et du modèle de détection	18
4.7.1 Annotation des nouvelles images	18
4.7.2 Intégration au dataset existant	19
4.7.3 Amélioration du modèle existant	19
4.7.4 Développement du nouveau modèle 360	19
4.8 Association image/position sur carte	21

5	Gestion de projet	22
5.1	Démarrage et contexte initial	22
5.2	Organisation de l'équipe	22
5.3	Calendrier et jalons	23
5.4	Méthode de travail et outils	23
5.5	Gestion des risques	23
5.6	Transmission inter-années	24
6	Résultats	24
6.1	Infrastructure logicielle	24
6.2	Acquisition d'images	25
6.3	Performances IA	25
6.4	Navigation autonome	26
6.5	Cartographie	26
6.6	Géolocalisation des fissures	27
6.7	Tableau de synthèse	27
7	Conclusion et Ouvertures	27
7.1	Conclusion	27
7.2	Ouvertures	28
	Références	29

1. Introduction

Ce projet industrie est réalisé en collaboration avec l'ANDRA (Agence nationale pour la gestion des déchets radioactifs) [1], une institution publique chargée du stockage des déchets nucléaires sur le moyen et long terme. Dans un premier temps, l'ANDRA a été chargée de développer un système de stockage en surface pour les déchets de faible et moyenne activité, en reprenant la gestion du Centre de stockage de la Manche et en établissant rapidement des règles pour encadrer et sécuriser ce stockage. Par la suite, l'agence a ouvert plusieurs sites dédiés à ce type de déchets, atteignant en 2020 un volume total de 1,3 million de mètres cubes stockés.

En parallèle, l'ANDRA a conduit plus de deux décennies de recherches sur le stockage géologique en profondeur. Ces travaux ont abouti au projet CIGEO, un site de stockage en profondeur, à environ 500 mètres sous terre, dans une couche géologique profonde, située dans la région de Bure, à cheval entre la Meuse et la Haute-Marne.

Cette infrastructure d'envergure comptera, dans sa version finale, plus de 270 kilomètres de galeries et d'alvéoles, avec une capacité de stockage d'environ 80 000 mètres cubes de déchets à haute activité et à longue durée de vie.

Cependant, en raison du cisaillement entre les jointures de la galerie, des fissures peuvent apparaître et se développer. Les galeries s'étendant sur plusieurs kilomètres, suivre l'évolution des fissures est une tâche répétitive et chronophage.

C'est dans ce contexte qu'est né le projet de ce parcours industrie, porté sur le thème suivant : “ Robotique et IA en environnements complexes : exploration de la galerie souterraine de l'ANDRA avec un robot et détection avancée de fissures.”



FIGURE 1 – Projet Cigéo, vue du laboratoire à Bure (Meuse)

2. État de l'art et du projet

2.1 Détection de fissures

Les technologies classiques de détection de fissures reposent majoritairement sur la segmentation par IA. Toutefois, la variabilité des propriétés des murs ou de l'environnement en lui même (crépi, béton lisse ou luminosité) rend l'usage de bases de données « universelles » inefficace pour le site spécifique de l'ANDRA.

Jusqu'en 2024, les tentatives d'utilisation de modèles open data (comme le *crack Dataset* [2]) se sont soldées par des échecs en production : les câbles, tuyaux et interstices des galeries étaient systématiquement confondus avec des fissures (voir figures 2 et 3).

Avancées 2024-2025 : Pour pallier ce problème, un jeu de données spécifique à l'environnement de Bure a été constitué, comportant près de 300 images annotées manuellement issues de la caméra PTZ du robot. L'entraînement d'un modèle YOLOv11 [3] sur cette base a permis d'obtenir des performances satisfaisantes, éliminant la majorité des faux positifs liés aux infrastructures (câbles, barrières).

Les limitations actuelles de ce système sont :

- La nécessité pour le robot d'être à l'arrêt lors de la prise de vue (sensibilité aux vibrations provoquées par la navigation).
- Une distance au mur qui doit être inférieure à deux mètres pour garantir la résolution nécessaire à la détection.
- Une couverture de l'ensemble de l'arche du tunnel qui reste à perfectionner.



FIGURE 2 – Un interstice



FIGURE 3 – Une barrière

2.2 Positionnement et navigation

Le positionnement en milieu souterrain reste le défi majeur. Si les robots haut de gamme utilisent des solutions onéreuses combinant LiDAR 3D et centrales inertielles de précision pour éviter le dérapage (« drift »), notre robot repose lui sur un Agilex Scout Mini [4, 5] équipé d'un LiDAR 2D [6, 7] et d'une caméra stéréoscopique ZED2i [8, 9].

Depuis 2025, l'architecture logicielle est entièrement portée sur le framework **ROS2** [10], standard industriel actuel, abandonnant les API propriétaires précédentes pour gagner en modularité.

Limites du SLAM et solution AMCL : Les travaux récents ont démontré que l'approche par SLAM (Simultaneous Localization and Mapping) [11] via un filtre de Kalman étendu (EKF) [12] entraînait une dérive trop importante avec les capteurs actuels, rendant la cartographie autonome difficile sur de longues distances.

L'état actuel de la solution de navigation repose donc sur l'algorithme AMCL (Adaptive Monte Carlo Localization) [13]. Cette méthode, validée lors des tests sur site en mai 2025, utilise une carte statique préétablie de la galerie (fichiers `.pgm`). Elle permet de recalibrer la position du robot grâce aux données du LiDAR 2D, compensant ainsi les erreurs d'odométrie et offrant une localisation fiable pour le placement des fissures détectées sur la carte et pour la continuité de la navigation, en s'appuyant sur Nav2 [14].

3. Semestre 1

3.1 Prise en main du robot

3.1.1 Présentation des parties indépendantes

Notre robot est constitué de 5 parties indépendantes que nous allons détailler ci-dessous.

Caméra PTZ Une caméra Pan-Tilt-Zoom motorisée Marshall CV-605 [15] offrant une rotation panoramique de $\pm 135^\circ$, une inclinaison de $\pm 90^\circ$ et un zoom adaptatif en temps réel. Il s'agit du même modèle que celui des années précédentes, offrant une résolution de 1080p à 60 fps, mais elle ne dispose pas de stabilisateur d'image intégré. Il est donc crucial de minimiser les vibrations pendant son utilisation pour garantir la netteté de l'image. Autrement dit, le robot devra se déplacer à une vitesse modérée, ce qui ne pose pas de problème en soi, mais doit être pris en compte pour assurer des prises de vue claires et précises.

LiDAR 2D Un LiDAR YDLidar TG15 [6, 7] (Light Detection and Ranging) qui utilise des impulsions laser pour mesurer des distances avec une grande précision. En émettant des faisceaux lumineux et en calculant le temps qu'ils mettent à revenir après avoir réfléchi sur des objets, ce genre de capteur est capable de reconstituer en deux dimensions l'environnement, même dans des conditions de faible luminosité.

ZED2i Cette caméra Stereolabs ZED2i [8, 9] est une caméra de profondeur, ce qui lui permet d'estimer si le robot se déplace en avant, en arrière ou s'il effectue une rotation. Ces informations seront essentielles lors de la phase de localisation. De plus, cette caméra possède une unité de mesure inertielle (IMU), un capteur capable de mesurer les accélérations linéaires et les vitesses angulaires grâce à des accéléromètres et des gyroscopes. L'IMU permet ainsi d'estimer la position, l'orientation et les mouvements du robot dans l'espace.



FIGURE 4 –
PTZ Marshall CV-605



FIGURE 5 –
Ydlidar TG15



FIGURE 6 –
Stereolabs ZED2i

Jetson Orin Nano Microcontrôleur NVIDIA Jetson Orin Nano [16] équipé d'un GPU intégré, offrant ainsi une capacité de traitement suffisamment puissante pour exécuter des tâches complexes, telles que la reconnaissance basée sur l'intelligence artificielle, directement sur le robot. Cette architecture permet d'éviter le recours à des solutions de cloud computing, qui pourraient introduire des latences et seraient inadaptées à un environnement enfoui à 500 m de profondeur.

Agilex Scout Mini Enfin, nous disposons d'une base mobile Agilex Scout Mini [4, 5] adaptée au terrain accidenté et pouvant se contrôler à l'aide d'une télécommande ou à partir d'un microcontrôleur connecté à un bus CAN.



FIGURE 7 – Jetson Orin Nano



FIGURE 8 – Agilex Scout Mini

3.1.2 Remise en service et configuration matérielle

Contrairement à l'année dernière, l'équipe 2025-2026 n'a pas eu à construire la tourelle du robot, mais nous avons dû reconstituer l'architecture logicielle suite à la perte de l'image système du robot. La priorité a donc été de rétablir la communication entre l'unité de calcul (**Jetson Orin Nano** [16]) et les différents actionneurs et capteurs.

Premièrement, la connexion avec la base mobile **Agilex Scout Mini** [5] a nécessité une reconfiguration de l'interface CAN. Une modification du code source du driver `scout_base` a été nécessaire pour permettre la reconnaissance de l'interface nommée `agilex` (au lieu des interfaces standards `can0`), utilisée par le systemd des ingénieurs du TechLab, assurant désormais un pilotage fluide et la remontée des données odométriques sur le topic `odom_robot`.

Par la suite, l'intégration des capteurs a présenté des résultats contrastés :

- **Caméra ZED2i** : La caméra est fonctionnelle et publie correctement les données visuelles et inertielles (IMU), qui ont été intégrées à la configuration du filtre de Kalman étendu (EKF) [12]. Le package `zed_wrapper` [9] est installé et la caméra est détectée, mais les tests ne sont, pour l'instant, pas concluants.
- **LiDAR 2D** : Bien que la connexion série soit établie et validée sur le port `/dev/ttyTHS1`, le capteur rencontrait à cette période des erreurs d'initialisation empêchant le démarrage du scan. Des tests avec différents fichiers de configuration et différents baudrates ont été effectués sans succès, laissant penser qu'il s'agissait d'une panne matérielle ou d'un problème de connexion physique.
- **Caméra PTZ** : La caméra était dans un premier temps inaccessible sur le réseau. Mais elle est désormais accessible via son adresse IP statique grâce à un script de configuration réseau. Les images sont capturées via RTSP et publiées correctement, et le contrôle PTZ est fonctionnel via le protocole VISCA over IP [15].

Cette phase de remise en service a permis de valider une architecture modulaire, où chaque capteur peut être activé ou désactivé via des arguments de lancement, garantissant la continuité des développements malgré les maintenances matérielles en cours.

3.1.3 Projet parallèle : Vision à 360 degrés

En marge de la remise en état fonctionnelle du robot, un axe de développement parallèle a été initié pour l'année 2025-2026 : l'intégration d'une caméra 360°. Cette initiative vise à pallier les limitations de la caméra PTZ actuelle, qui nécessite des arrêts fréquents et un ciblage précis pour obtenir des images nettes des parois.

L'objectif de ce projet est de permettre une analyse plus fluide et exhaustive des galeries :

- **Acquisition globale** : Contrairement aux caméras classiques, la technologie 360° permet de capturer l'ensemble de la voûte et des murs en une seule prise.
- **Détection continue** : Ce matériel est destiné à être couplé avec un nouveau modèle d'intelligence artificielle (entraîné spécifiquement sur ces images panoramiques) pour effectuer une détection de fissures en continu pendant les rondes, sans interrompre la navigation du robot.

Ce module constitue une évolution majeure par rapport aux années précédentes et fait l'objet d'un développement spécifique (entraînement IA et tests d'acquisition) mené conjointement à la restauration du robot.

3.2 Reprise des travaux de l'année 2024-2025

Bien que ce projet industriel avec l'ANDRA [1] soit mené pour la cinquième année consécutive, la continuité entre les équipes reste le défi majeur. Contrairement aux années précédentes où la documentation faisait défaut, nous avons pu bénéficier cette année d'un accès complet aux ressources numériques produites par l'équipe 2024-2025 : le dépôt GitHub contenant l'architecture ROS2 [10] et le Drive hébergeant le modèle d'intelligence artificielle YOLOv11 adapté à la caméra PTZ, accompagné de ses photos d'entraînement.

Cependant, la reprise a été fortement ralentie par un incident technique critique : l'image système du robot, contenant l'ensemble de l'environnement configuré, a été supprimée sans sauvegarde. Cette perte nous a contraints à ne pas simplement « utiliser » le travail précédent, mais à devoir le comprendre en profondeur pour le reconstruire (réinstallation des drivers, configuration des espaces de travail, création des scripts de compilation).

Critique de l'architecture historique : L'analyse menée par l'équipe 2024-2025 a mis en évidence les limites méthodologiques des approches antérieures. La stratégie employée en 2023-2024 reposait sur la détection conjointe des fissures et d'étiquettes calibrées présentes dans les tunnels, utilisant ces dernières comme référentiel pour dimensionner la taille des fissures. Cette méthode par comparaison s'est avérée peu viable en raison d'une incertitude trop importante (± 4 cm) sur la taille des fissures et d'un manque de robustesse, l'algorithme confondant fréquemment les fissures avec des câbles ou des interstices, comme le montrent les figures 2 et 3 (pour plus de détails, cf. section "Reprise du code de l'année dernière" du rapport 2024-2025).

L'équipe 2024-2025 avait donc pris la décision d'abandonner ce protocole et ce modèle contraignant au profit d'un nouveau modèle de détection par réseaux de neurones. Nous validons et poursuivons cette stratégie technologique pour l'année 2025-2026 :

- **Conservation de YOLO :** Les tests préliminaires montrent que ce modèle est robuste aux faux positifs (câbles, barrières). Nous avons ainsi récupéré le fichier de poids `best.pt` pour l'intégrer à notre nouvelle stack logicielle.
- **Transition vers la vision 360° :** Si l'algorithme de détection est conservé, nous prévoyons de le réentraîner pour qu'il s'adapte aux images grand angle de la nouvelle caméra 360°, afin de s'affranchir définitivement des contraintes de mouvement de la caméra PTZ.

3.3 Préparation de la première descente

Notre première descente au laboratoire de Bure, réalisé le vendredi 6 février 2026, s'inscrit dans la continuité de la phase de reconstruction technique menée tout au long du premier semestre. Suite à la suppression accidentelle de l'image système du robot de l'année précédente, nous avons dû redéfinir nos priorités. L'objectif premier étant de rétablir une base de travail saine en recréant intégralement l'infrastructure logicielle (scripts d'installation, configuration ROS2 [10]) et en validant les pilotes des capteurs (`scout_base` [5] pour l'odométrie des roues, `zed_wrapper` [9] pour les données IMU, et `ydliar_ros2_driver` [7] pour la cartographie et la détection d'obstacles).

3.3.1 Outillage et reproductibilité

Pour garantir que chaque membre de l'équipe puisse mettre en service le robot de manière autonome, une série de scripts standardisés a été développée :

- `scripts/setup.sh` : installation de l'ensemble des dépendances ROS2 et drivers ;
- `scripts/build.sh` : compilation des espaces de travail ROS2 ;
- `scripts/launch.sh` : lancement du système complet avec options modulaires.

Ces options de désactivation permettent de tester le système même lorsqu'un composant est en maintenance, assurant la continuité du développement. En complément, plus d'une dizaine de guides Markdown ont été rédigés tout au long de l'année : démarrage du robot, connexion réseau (TechLab et hotspot terrain), débogage, utilisation de RViz2, arbre TF, détection YOLO, procédures de navigation et sauvegarde de l'image disque.

3.3.2 État du robot avant la descente

À l'approche de cette première descente, l'état du robot est le suivant :

- **Base mobile Agilex Scout Mini** : Fonctionnelle. Le driver `scout_base` [5] communique avec le robot via l'interface CAN `agilex` et publie correctement les données odométriques sur `/odom_robot`.
- **Caméra PTZ Marshall CV-605** : Fonctionnelle. La caméra est accessible via son adresse IP statique grâce au script `ptz-network-setup.sh`. Le contrôle est opérationnel via le protocole VISCA over IP [15], et les nœuds de séquence automatique (`sequence_photo` et `sequence_video`) sont en place.
- **Caméra ZED2i** : Partiellement fonctionnelle. Le package `zed_wrapper` [9] est installé et la caméra est détectée, elle publie les données visuelles et inertielles (IMU). Cependant, la cohérence des données renvoyées n'a pas encore été vérifiée.
- **LiDAR 2D** : Non résolu. Le diagnostic a identifié un port micro USB arraché sur la carte Radar_Con. Un branchement de secours via USB-C pour assurer le rôle de ce branchement arraché va être tenté. (cf. figure 10)
- **Hotspot Jetson** : Configuré et validé. Le robot, équipé de sa propre carte réseau, crée automatiquement un réseau Wi-Fi `JetsonWIFI` lorsqu'il ne capte pas le réseau du TechLab, permettant la connexion SSH en galerie via `ssh techlab@orin2.local`.

3.3.3 Objectifs de la descente

L'objectif principal de cette échéance de février est double :

1. **Validation système** : Reproduire les résultats de l'équipe 2024-2025 en conditions réelles avec le robot fraîchement reconfiguré, notamment sa capacité à effectuer des cycles simples (avancer, s'arrêter, prendre une photo).
2. **Acquisition de données 360°** : Capturer un nouveau jeu de données à l'aide d'une caméra 360° prêtée par l'ANDRA. Cette acquisition est indispensable pour entraîner notre nouveau modèle de détection afin qu'il puisse, à terme, analyser l'ensemble de la voûte et non plus uniquement les murs verticaux.

Cette descente permettra ainsi de vérifier la cohérence entre l'environnement de simulation du TechLab et la réalité des tunnels, tout en fournissant à l'équipe IA les données nécessaires à l'amélioration du modèle de détection.



FIGURE 9 –
Ricoh Theta 360

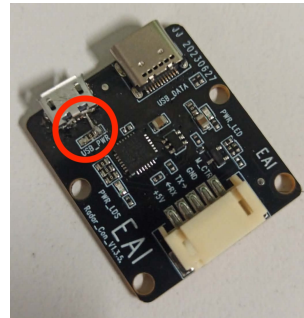


FIGURE 10 –
Chipset LiDAR cassé

Contraintes et limitations : Malgré ces avancées, plusieurs défis subsistent à l’approche de cette première descente dans les tunnels de l’ANDRA :

- **Absence du LiDAR :** Le capteur LiDAR n’étant pas encore fonctionnel, la navigation se fera sans cartographie ni détection d’obstacles laser, limitant les tests de navigation aux séquences de prise de photos ou vidéos.
- **Disponibilité du matériel 360° :** L’équipe ne dispose pas de sa propre caméra 360° intégrée au robot. Pour cette session d’acquisition, nous nous appuyerons sur un équipement fourni ponctuellement par l’ANDRA.
- **Données ZED2i non vérifiées :** La cohérence des données inertielles et visuelles de la caméra ZED2i n’a pas encore été validée, ce qui limite la fiabilité du filtre de Kalman étendu [12] pour le positionnement (visible dans RViz).

3.4 Résultats de la première descente

La première descente dans les galeries de l’ANDRA s’est déroulée le vendredi 6 février 2026. Cette intervention a permis de confronter pour la première fois le système reconstruit aux conditions réelles du site de Bure.

3.4.1 Succès opérationnels

L’ensemble du système logiciel s’est montré fonctionnel en conditions réelles :

- **Séquence de prise de photos :** La séquence automatique de capture via la caméra PTZ a fonctionné comme prévu. Le robot a pu effectuer des cycles complets (avancer, s’arrêter, prendre des photos) dans une galerie de longueur suffisante.
- **Hotspot opérationnel :** Le système de hotspot de la Jetson s’est activé automatiquement en l’absence du réseau du TechLab, permettant la connexion SSH et le contrôle à distance du robot depuis un PC portable.
- **Nœuds ROS2 fonctionnels :** L’ensemble des nœuds ROS2 déployés ont fonctionné correctement, confirmant la robustesse de l’architecture reconstruite.
- **Acquisition du dataset 360° :** Grâce à la caméra 360° prêtée par l’ANDRA, un jeu de données panoramiques a été capturé dans les galeries. Ce dataset constitue la base de travail pour l’entraînement du futur modèle de détection adapté aux images grand angle.

3.4.2 Incident matériel

Un incident est survenu lors de la descente : l'alimentation de la Jetson Orin Nano a été arrachée suite à une mauvaise manipulation. Cette panne a interrompu les tests en cours. Cependant, la carte a été réparée par Paul-Antoine le week-end suivant, avec un changement du connecteur d'alimentation, rendant le robot de nouveau opérationnel la semaine suivante.

3.4.3 Constats et axes d'amélioration

La descente a également mis en lumière plusieurs points à améliorer :

- **Couverture PTZ insuffisante** : La séquence actuelle de 3 photos ne couvre pas l'intégralité de l'arche de la galerie. Plusieurs solutions sont envisagées : passer à une séquence de 5 photos, revoir l'enchaînement des positions PTZ ou utiliser le script vidéo à très basse vitesse.
- **Trajectoire RViz non planaire** : Les premières visualisations dans RViz2 ont révélé que la trajectoire du robot ne s'affiche pas sur un plan horizontal, indiquant un problème dans les transformations TF à investiguer.

Cette première descente a permis de valider les briques logicielles fondamentales du système et d'identifier clairement les axes de travail prioritaires pour le second semestre.

4. Semestre 2

Le second semestre s'ouvre sur les enseignements tirés de la première descente et vise à faire progresser le robot vers un système réellement exploitable en galerie. Les travaux se sont articulés autour de quatre axes : la fiabilisation des capteurs, l'amélioration des performances d'inférence de la Jetson, la validation de la navigation sur carte, puis la fusion entre navigation autonome et acquisition d'images.

4.1 Résolution du problème LiDAR

Le problème LiDAR, identifié depuis le premier semestre, a été définitivement résolu le 18 février 2026. Le diagnostic avait révélé que le port micro USB de la carte Radar_Con était arraché, empêchant l'alimentation du moteur nécessaire à la rotation continue du LiDAR. Un branchement de secours via le port USB-C s'est avéré fonctionnel.



La résolution a nécessité deux corrections :

- **Changement de port** : Passage de `/dev/ttyTHS1` à `/dev/ttyUSB0` pour exploiter le branchement USB-C.
- **Correction de la configuration** : L'ancien fichier de configuration (`G4.yaml`) était inadapté au modèle réel du capteur. Le LiDAR est un TG15 [6], et l'adoption du fichier officiel `TG.yaml` du package `ydliar_ros2_driver` [7] a permis un fonctionnement normal.

4.2 Optimisation YOLO sur Jetson

Afin d'exploiter pleinement le GPU de la Jetson Orin Nano [16] pour la détection de fissures, une chaîne d'optimisation complète a été mise en place. Une image Docker spécialisée (`Dockerfile.jetson`), basée sur `dustynv/14t-pytorch:r36.2.0`, a été construite pour isoler l'environnement de détection.

Cette image intègre ROS 2 [10], PyTorch avec CUDA et la bibliothèque Ultralytics [3], garantissant la compatibilité GPU native sur la plateforme Jetson.

Un script de conversion (`scripts/convert_to_tensorrt.sh`) permet de transformer le modèle PyTorch `best.pt` en un moteur TensorRT `best.engine` [17], offrant une accélération de l'inférence de l'ordre de 3 à 5 fois par rapport au modèle PyTorch. Le code du nœud `image_subscriber` a été adapté pour détecter et utiliser automatiquement le moteur TensorRT s'il est disponible, avec replis PyTorch CPU si la mémoire GPU est insuffisante. Grâce à ce gain de performance, le nœud charge désormais le modèle en mode segmentation, publie la pose sur `/position_detectee` à chaque fissure détectée, et trace le contour des fissures plutôt que de simples boîtes englobantes. (cf. figure 20)

L'ensemble est orchestré par le script `docker/launch_jetson.sh`, qui lance le conteneur avec la configuration réseau host, le montage des volumes nécessaires et la vérification automatique des dépendances.

4.3 Travaux préparatoires avant la seconde descente

Entre la première et la seconde descente, une phase de stabilisation a été menée sur la chaîne navigation/cartographie afin de corriger les incohérences observées en février.

4.3.1 Révision de la séquence PTZ

Avant la seconde descente, un travail spécifique a été mené sur les séquences de capture PTZ. Les essais de février avaient montré que la séquence initiale à 3 photos ne couvrait pas l'ensemble de l'arche et restait assez lente à l'exécution.

Ainsi, les séquences ont été révisées et les modifications ont porté sur :

- le passage de 3 prises de photo à 5 prises ;
- la réorganisation des positions pan/tilt pour améliorer la vitesse des séquences ;
- l'évaluation du script vidéo à très basse vitesse comme alternative de capture continue avec la création d'un nouveau type de séquence (`sequence_video`).

4.3.2 Reconstruction de l'arbre TF

Les visualisations RViz mettaient en évidence une trajectoire non planaire, signe d'un problème de cohérence des transformations. Le travail effectué a porté sur :

- la reprise complète des frames et transformations statiques ;
- l'intégration explicite des repères publiés par `scout_base` [5] ;
- l'ajustement des distances inter-capteurs dans les TF statiques ;
- la suppression de la frame `world`, redondante avec `odom`.

4.3.3 Amélioration de la qualité des scans LiDAR

Pour la cartographie, une configuration dédiée `ydliidar_TG15.yaml` a été consolidée à partir de la documentation YDLidar [6], puis une plateforme imprimée en 3D a été utilisée pour libérer le LiDAR des perturbations causées par les piliers de la tourelle. Ainsi, deux modes de montage ont été conservés selon l'usage (cartographie ou navigation), avec adaptation du paramètre `laser_mount_yaw` :

- $\pi/2$ pour le montage original (hérité de l'année dernière) ;
- 0.0 pour la création de carte en SLAM (LiDAR aligné avec l'avant du robot).

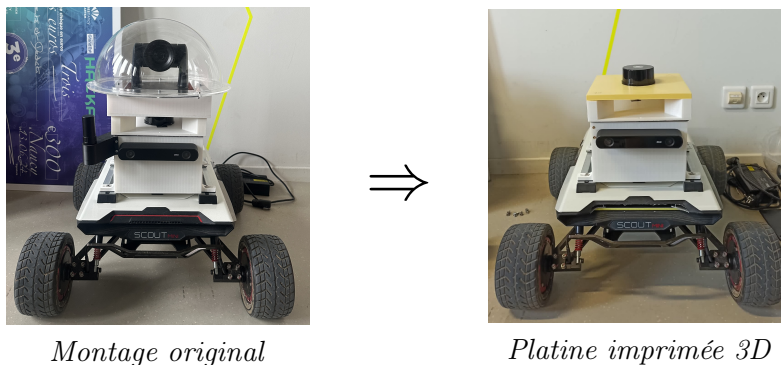


FIGURE 11 – Évolution du montage du LiDAR pour la cartographie.

4.3.4 Mise à jour EKF + SLAM

La fusion d'état a été simplifiée pour gagner en robustesse, en s'appuyant sur le package `robot_localization` [12] :

- la position et la vitesse proviennent maintenant uniquement de `/odom_robot` ;
- l'IMU ZED [9] est conservée uniquement pour `vyaw` (orientation du robot) ;
- la VO ZED n'est plus fusionnée pour éviter les conflits avec l'odométrie des roues ;
- le SLAM Toolbox [11] est maintenant paramétré avec des frames strictes :
 - `map` = repère global de la carte ;
 - `odom` = repère local continu pour l'odométrie ;
 - `base_link` = repère du robot.
- l'EKF publie donc maintenant l'état du robot dans les bons repères et SLAM utilise exactement les mêmes noms de frames pour une parfaite cohérence ;
- il n'y a plus de frames parasites ni de doublons TF qui contredisent cette chaîne.

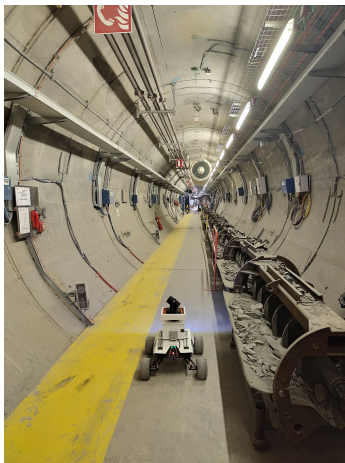
4.3.5 Sauvegarde de l'image disque

Tirant les enseignements de la perte d'image système en début d'année, une image complète du disque NVMe du robot a été réalisée à l'issue de la seconde descente, selon la procédure décrite dans `docs/BACKUP.md`. L'archive est vérifiable via une empreinte SHA256 conservée dans le dépôt. Une clé USB contenant cette image sera transmise avec le robot à l'équipe 2026-2027, afin qu'une restauration complète reste possible en cas de panne matérielle ou logicielle grave.

4.4 Résultats de la seconde descente

La seconde descente a constitué le jalon principal du semestre 2. Elle a permis de valider en conditions réelles une version nettement plus mature de la stack logicielle.

- **Séquences de capture améliorées** : la nouvelle séquence photo couvre mieux l'arche de la galerie et réduit le temps de capture par arrêt.
- **Séquence vidéo opérationnelle** : l'acquisition continue vidéo est fonctionnelle, mais reste encore assez lente pour assurer une bonne qualité de capture.
- **Pipeline GPU validé** : le traitement des images via le GPU Jetson [16] (Docker + TensorRT [17]) fonctionne en situation réelle et est bien plus rapide.
- **Acquisition de données 360°** : un jeu de données supplémentaire a été collecté grâce à la caméra 360° prêtée par l'ANDRA.
- **Cartographie initiale encourageante** : une première tentative de cartographie des tunnels a été réalisée, mais reste insatisfaisante, car la dérive reste encore trop importante pour un la création d'une carte fiable.



Le robot en mission



Première cartographie

FIGURE 12 – Seconde descente

4.5 Navigation autonome sur carte

Le semestre 2 a aussi marqué la transition vers un premier prototype de navigation autonome orientée mission avec des waypoints, basée sur AMCL [13] et Nav2 [14]. Le point central de cette transition est l'usage de RViz comme interface de création et de validation des trajectoires, en particulier via 2D Nav Goal.

- **Validation AMCL + Nav2** : lancement en mode carte (`map_server` + `amcl`) et validation de l'envoi d'objectifs de navigation dans RViz.
- **Nœud de mission** : création du nœud `trajectoire_mission.py`, qui lit une trajectoire YAML puis envoie séquentiellement des goals `navigate_to_pose`.
- **Trajectoires construites depuis RViz** : les scripts s'appuient sur les objectifs posés manuellement avec 2D Nav Goal. Chaque pose est récupérée, puis l'ensemble est enregistré dans un YAML de waypoints.



FIGURE 13 – Visualisation RViz2

4.6 Fusion navigation + acquisition

L'objectif a ensuite été de fusionner le déplacement autonome et la capture de données. Cette fusion a nécessité une réflexion conjointe sur l'orchestration logicielle et sur le montage mécanique des capteurs. Un package ROS2 [10] dédié (`patrouille_autonome`) a été créé pour cette fonction.

4.6.1 Compromis mécanique LiDAR/PTZ

Ce travail s'est accompagné d'une réflexion sur l'architecture mécanique de la tourelle. Les premiers essais ont confirmé que le LiDAR était un capteur clé, non seulement pour la cartographie, mais aussi pour la navigation autonome. Or, la configuration initiale dégradait fortement les scans (masqués par les piliers de la tourelle), ce qui nuisait directement à la qualité de la localisation, aussi bien en phase de cartographie qu'en phase de navigation.

Une refonte complète de la tourelle a d'abord été envisagée, avec un habillage en plexiglas autour du LiDAR tout en gardant la caméra PTZ placée au-dessus. Cette solution a finalement été abandonnée, les tests ayant montré qu'aucun matériau ne laissait suffisamment passer les scans. Il était donc impossible de conserver cette structure horizontale avec la caméra PTZ au sommet de la tourelle.

La stratégie retenue repose donc sur un montage plus pragmatique : positionner la caméra derrière la tourelle sur une platine imprimée en 3D pour cet usage, et conserver le LiDAR dans une position dominante sur une nouvelle platine dédiée imprimée également en 3D, afin de lui garantir un champ de scan aussi dégagé que possible. L'objectif est d'exploiter au maximum la précision du LiDAR pour la cartographie et la navigation, tout en maintenant la capacité de capture d'images de l'arche d'un tunnel.

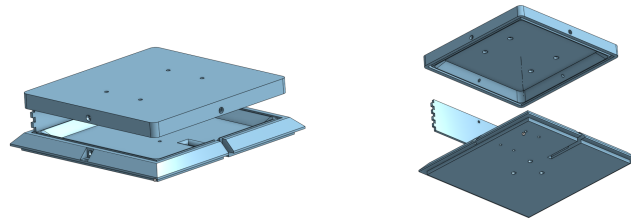


FIGURE 14 – Premier prototype avec plexiglas



FIGURE 15 – Montage final

Ce nouveau positionnement a aussi permis de simplifier la configuration logicielle : le paramètre `laser_mount_yaw`, qui compensait auparavant l'angle de montage du LiDAR par rapport à l'axe du robot, n'est plus nécessaire. Le LiDAR étant maintenant aligné avec l'avant du robot sur sa platine dédiée, la valeur par défaut est directement applicable, aussi bien pour la cartographie que pour la navigation.

4.6.2 Orchestration logicielle

Une fois le déplacement et l'acquisition stabilisés séparément, l'enjeu est de les enchaîner de manière fiable sur le terrain : lancer les séquences de prise de vue sans bloquer la progression entre waypoints, tout en conservant un comportement de navigation acceptable face aux obstacles et en garantissant un arrêt propre en fin de mission. Les choix suivants matérialisent cette orchestration dans le package `patrouille_autonome`.

- **Orchestration photo/navigation** : script `fusion_photo_navigation` pour exécuter une mission de waypoints en parallèle d'une séquence photo.
- **Orchestration vidéo/navigation** : script `fusion_video_navigation` pour exécuter une mission de waypoints en parallèle d'une séquence vidéo.
- **Stratégie d'évitement d'obstacles** : les tests terrain ayant montré trop de faux positifs sur l'arrêt d'urgence, la stratégie retenue repose sur l'esquive native de Nav2 [14], jugée plus performante en mission réelle.
- **Fin de mission robuste** : arrêt automatique des séquences, envoi d'une commande pour stopper le mouvement et retour de la PTZ en position Home.

4.7 Amélioration du dataset et du modèle de détection

4.7.1 Annotation des nouvelles images

À l'issue de chaque descente, des images des galeries ont été collectées selon deux méthodes distinctes : via la caméra PTZ et via la caméra 360°. Ces deux sources produisent des images de nature très différente, ce qui a nécessité deux pipelines d'annotation adaptés.

Pour les images PTZ, l'annotation a été réalisée directement dans le logiciel Label Studio : chaque image a été analysée manuellement, et les fissures ont été délimitées par des polygones de segmentation.

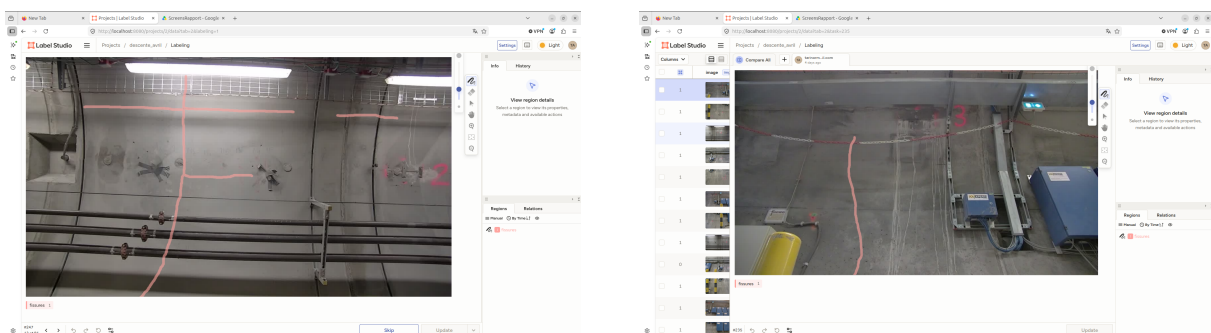


FIGURE 16 – Exemples d'images PTZ annotées sur Label Studio

Les images 360°, quant à elles, ne peuvent pas être annotées directement en raison des distorsions introduites par la projection équirectangulaire. Un prétraitement est donc appliqué en amont : l'image panoramique est dépliée sur les faces d'un cube, dont seules les faces droite, gauche et supérieure sont conservées, correspondant aux parois et à la voûte de la galerie. Cette mise à plat réduit significativement les déformations géométriques et rend l'annotation fiable.

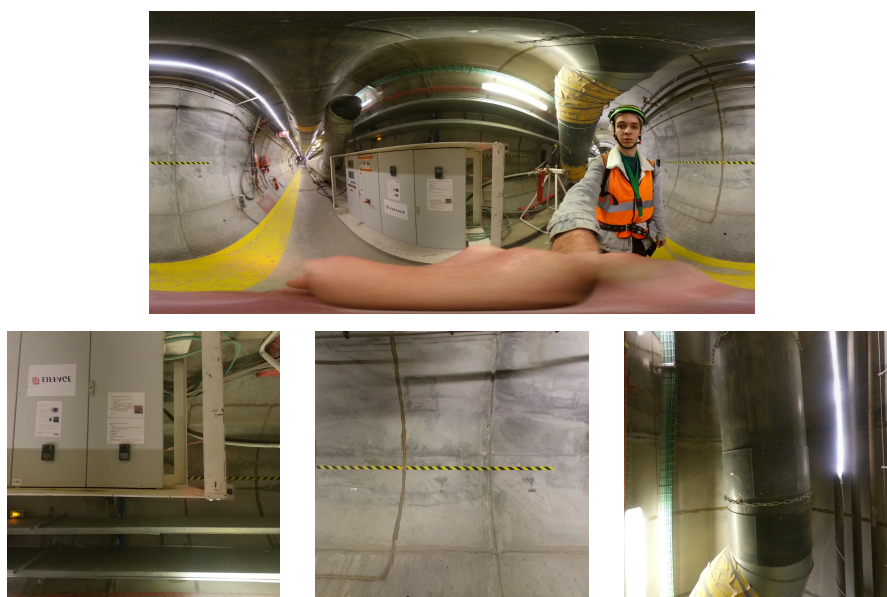


FIGURE 17 – Image 360° brute et les trois faces extraites après prétraitement

4.7.2 Intégration au dataset existant

L'équipe 2024-2025 nous a transmis un Drive contenant les images d'entraînement ainsi que les poids de leur meilleur modèle (**best.pt**), mais sans script d'entraînement ni organisation claire : les métriques étaient dispersées et aucune convention de nommage n'avait été établie. Une phase de restructuration a donc été nécessaire avant toute intégration de nos nouvelles images.

Ce travail a consisté à homogénéiser les formats d'annotation, à vérifier la cohérence des étiquettes entre les images héritées et celles issues de nos descentes, puis à fusionner les deux ensembles en un dataset unifié compatible avec l'entraînement YOLOv11.

Un nouveau Drive organisé a été constitué à l'issue de ce travail, destiné à être transmis à l'équipe 2026-2027. Il comprend le dataset consolidé, les scripts d'entraînement et une documentation claire, dans l'objectif de faciliter la reprise et l'extension du modèle.

Au cumulé, le dataset PTZ compte **451 images**, dont **304** avec fissures et **147** sans. Pour la caméra 360°, nous disposons de **344 images** après prétraitement cubemap, dont **214** avec fissures et **130** sans.

4.7.3 Amélioration du modèle existant

La restructuration du dataset hérité de l'équipe 2024-2025 s'est avérée plus longue que prévu. Les annotations des images héritées et celles issues de nos descentes utilisaient des formats et des conventions de nommage différents, ce qui a nécessité un travail d'homogénéisation important avant toute tentative de fusion. La fusion complète des deux corpus en un jeu d'entraînement commun n'a pas été finalisée dans le temps imparti ; en revanche, le dataset PTZ consolidé a pu être unifié en format compatible YOLOv11, trop tardivement pour lancer un nouvel entraînement cette année.

Le modèle PTZ opérationnel reste donc le **best.pt** transmis par l'équipe 2024-2025, intégré à la stack TensorRT décrite en section 4.2. Ce modèle, entraîné sur environ 300 images, continue de donner des résultats satisfaisants sur les images PTZ des galeries de type GER, comme le montrent les détections obtenues lors de la seconde descente (figure 20). Ce dataset de 451 images est disponible sur le Drive partagé et constitue une première priorité pour l'équipe 2026-2027, qui disposera d'une base significativement plus large que celle utilisée jusqu'ici.

4.7.4 Développement du nouveau modèle 360

Un nouveau modèle de segmentation **YOLOv11s-seg** a été développé spécifiquement pour les images issues de la caméra 360°. Après conversion cubemap, chaque image fait 1680×1680 pixels ; les fissures, très fines, n'y représentent que quelques dizaines de pixels de large. Les 610 images disponibles ont été annotées dans Label Studio (outil pinceau, export COCO converti en YOLO-seg), avec 367 tâches complétées au moment de l'entraînement, dont 233 contenant au moins une fissure. Après conversion et filtrage des contours trop petits, le dataset final comprend **344 images** dont **214 positives** et **130 négatives**.

Problème et solution Les premiers entraînements sur images plein format ont systématiquement produit des métriques nulles ($mAP_{50} = 0$) sur 100 epochs, avec une perte de validation (`val/seg_loss`) indéfinie (NaN). La cause identifiée est que les fissures deviennent trop petites après redimensionnement en entrée du réseau.

La solution retenue est une **découpe en tuiles** (*tiling*) : chaque image est subdivisée en tuiles de 640×640 px avec 20 % de chevauchement, les annotations étant reprojettées géométriquement sur chaque tuile. Cette opération multiplie le nombre d'exemples d'entraînement et augmente la taille apparente des fissures dans chaque tuile.

Résultats La `val/seg_loss` converge sans NaN dès la première epoch, et le modèle produit ses premières détections non nulles à partir de l'epoch 5. Après 100 epochs, le meilleur point de contrôle (`best.pt`, epoch 85) atteint les métriques suivantes :

Seuil conf.	Précision	Rappel	mAP_{50} (masque)
0,50	0,655	0,044	0,346
0,25	0,433	0,239	0,302
0,10	0,353	0,358	0,287
0,05	0,376	0,334	0,264

TABLE 1 – Métriques de `best.pt` selon le seuil de confiance. Le seuil 0,10 est retenu.

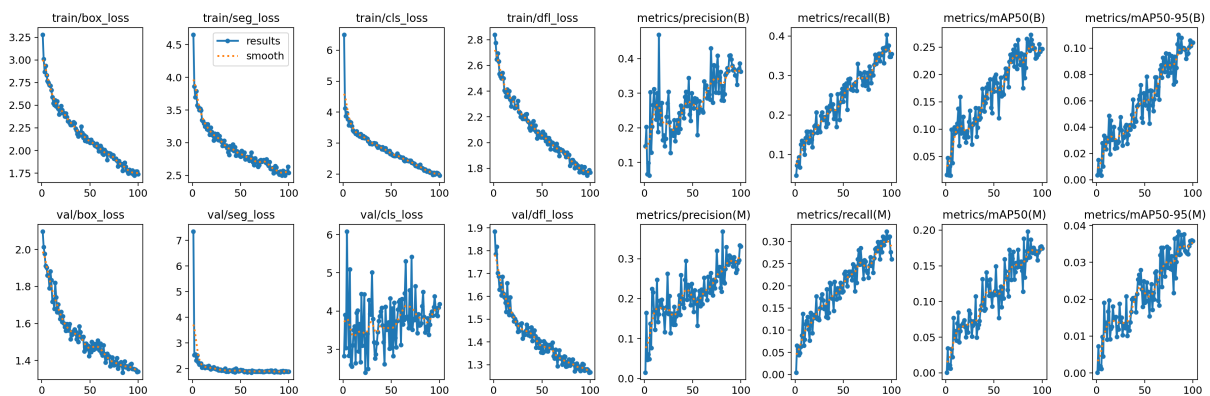


FIGURE 18 – Courbes d'entraînement (100 epochs). La `val/seg_loss` converge sans NaN et les métriques progressent de manière continue.

Le seuil de 0,10 est privilégié pour l'usage opérationnel : dans un contexte de surveillance structurelle, manquer une fissure est plus critique que produire une fausse alarme.

Perspectives Le rappel reste faible, principalement par manque de données annotées et par convergence incomplète (la perte d'entraînement n'est pas encore en plateau). Les pistes prioritaires pour l'équipe suivante sont : annoter les images restantes — idéalement en s'appuyant sur `best.pt` pour une pré-annotation automatique (*pseudo-labeling*) — et développer un pipeline d'inférence par tiling sur images complètes pour intégrer ce modèle au flux de traitement du robot.

4.8 Association image/position sur carte

La dernière brique logicielle de l'année consistait à relier détection et position : savoir où sur la carte se trouve chaque fissure détectée lors d'une prise de vue. Cette chaîne est aujourd'hui validée en laboratoire et clôt le périmètre fonctionnel défini pour 2025-2026.

Le nœud `image_subscriber` détecte les fissures sur les images reçues (`/photo_topic`) et, à chaque détection, publie la pose courante du robot sur `/position_detectee`.

Le nœud `report_fissures` s'abonne à ce topic et, pour chaque message :

- affiche la position en odom puis, après conversion TF, en map ;
- trace le point sur la carte et enregistre un PNG dans `ros2_ws/map_detections/`.

Les anciens nœuds de diagnostic `show_pos` et `position_publisher`, hérités des années précédentes et branchés sur des topics obsolètes, ont été retirés : leur rôle est couvert directement par `image_subscriber` et `report_fissures`.

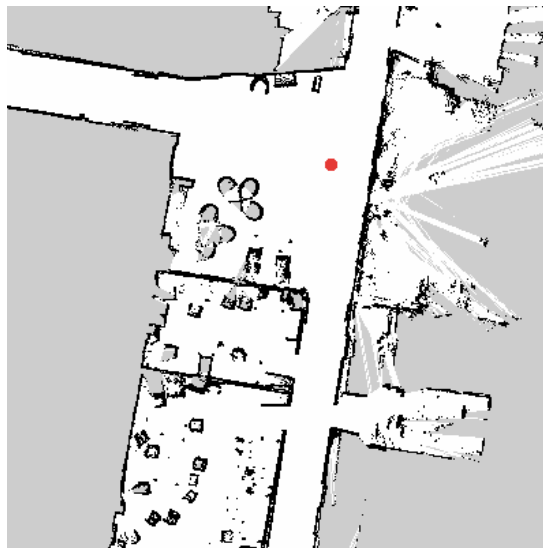


FIGURE 19 –
Exemple de position liée à une
détection

5. Gestion de projet

5.1 Démarrage et contexte initial

En raison d'aléas administratifs liés à l'affectation des sujets, le projet n'a débuté officiellement que fin novembre. Cette période de latence a toutefois été mise à profit pour nous former en autonomie, à la prise en main du modèle de détection YOLO. Anticipant une problématique centrée sur l'analyse d'image, cette montée en compétence nous avait été conseillée par nos encadrants.

Une fois le sujet attribué, celui-ci a révélé une composante robotique majeure que nous ne pouvions pas anticiper. Ne disposant pas de connaissances spécifiques dans ce domaine, la prise en main de l'aspect navigation et matériel — ROS2 [10], drivers capteurs et stack Nav2 [14] — a constitué le premier défi technique majeur de ce semestre.

Dans ce contexte, la priorité a été donnée à l'appropriation technique plutôt qu'à la mise en place de processus de gestion de projet complexes. Nous avons rapidement établi le contact avec l'équipe précédente pour récupérer l'historique des travaux (dépôt GitHub et Drive). Si l'accès aux ressources logicielles s'est fait sans encombre, l'accès physique au robot, situé au TechLab, a nécessité davantage de coordination logistique.

5.2 Organisation de l'équipe

Une fois le projet lancé, nous nous sommes répartis en deux binômes complémentaires, selon les compétences et affinités de chacun. Cette organisation a permis de travailler en parallèle sur deux axes techniques, afin d'éviter de bloquer l'avancement global.

Binôme IA — Lucas et Adrien. Lucas et Adrien ont pris en charge toute la chaîne de détection et d'analyse d'images :

- tri et annotation des jeux de données (images PTZ et panoramiques 360°) ;
- entraînement et évaluation du modèle YOLO sur images PTZ, puis développement d'un modèle YOLOv11s-seg adapté aux images panoramiques 360° ;
- comparaison des performances d'inférence entre PyTorch et TensorRT [17, 16] ;

Binôme robotique — Vincent et Paul-Antoine. Vincent et Paul-Antoine ont assuré la remise en service du robot, sa fiabilisation matérielle, la navigation autonome et son couplage avec les séquences d'acquisition :

- installation et configuration des drivers (`scout_base`, ZED, LiDAR, PTZ) ;
- développement des scripts de lancement et rédaction de la documentation ;
- conception des séquences photo et vidéo (`sequence_photo`, `sequence_video`) ;
- résolution des pannes matérielles (LiDAR, alimentation Jetson) ;
- conception et impression 3D des nouvelles platines LiDAR et PTZ ;
- implémentation de la navigation autonome via waypoints AMCL [13] + Nav2 [14] ;
- création d'une sauvegarde disque (procédure documentée dans `BACKUP.md`) ;
- finalisation de `report_fissures` : association de chaque fissure détectée à une position sur la carte (odom → map, tracé PNG).

5.3 Calendrier et jalons

Au-delà de la répartition des tâches entre binômes, le projet a suivi une progression temporelle marquée par des échéances décisives. Depuis la prise en main du robot fin novembre jusqu'à la validation de la chaîne image-position au printemps 2026, chaque période a concentré des enjeux techniques distincts : reconstruction de l'infrastructure, validation terrain, montée en autonomie, puis bouclage de la géolocalisation des détecteurs. Le tableau suivant synthétise ces cinq étapes en croisant, pour chaque jalon, l'objectif visé au moment de l'échéance et le bilan que nous en tirons aujourd'hui.

Jalon	Date	Objectif	Résultat
Démarrage officiel	Nov 2025	Prise en main	Réinstallation complète
Soutenance mi-parcours	23 Janv	Bilan S1	LiDAR en panne + Workflow
1 ^{ère} descente	6 Fev	Validation sys	Séquence PTZ + dataset 360°
2 ^{ème} descente	10 Avr	Nav autonome	Nav2 + cartographie partielle
Soutenance finale	12 Juin	Bilan annuel	11/11 nœuds + image/position

TABLE 2 – Calendrier du projet 2025-2026

5.4 Méthode de travail et outils

La coordination s'est organisée autour de deux ressources partagées : le dépôt GitHub pour le code, la documentation ROS2, et un Drive commun pour les modèles YOLO et les datasets trop volumineux pour Git.

Les scripts standardisés (`setup.sh`, `build.sh`, `launch.sh`) permettent à tout membre de l'équipe de lancer le robot sans connaissance préalable de l'architecture.

Les deux binômes se sont synchronisés régulièrement pour arbitrer les choix techniques, en s'appuyant sur les issues GitHub pour le suivi des tâches. La répartition claire entre axe IA et axe robotique a limité les dépendances bloquantes : chaque binôme pouvait avancer en parallèle, sur des tâches quasi indépendantes.

5.5 Gestion des risques

Trois problèmes majeurs se sont concrétisés au cours de l'année. Leur matérialisation et les mesures correctives apportées sont présentées ci-dessous.

Perte de l'image disque. En début d'année, l'image système du robot — contenant l'ensemble de l'environnement configuré par l'équipe précédente — a été supprimée sans sauvegarde. La réinstallation complète de tous les drivers et la reconfiguration de l'architecture ROS2 [10] ont mobilisé plusieurs semaines, voire plusieurs mois. En réponse, une image complète du disque NVMe a été réalisée en fin de projet et une clé USB de restauration sera transmise avec le robot.

Pannes matérielles. Deux incidents ont perturbé les plannings : le port micro USB arraché sur la carte du LiDAR (résolu par une soudure USB-C) et le connecteur d'alimentation de la Jetson arraché lors de la première descente (résolu en une semaine par Paul-Antoine). L'architecture modulaire `enable_*` du launch file a permis de maintenir le développement en parallèle malgré ces indisponibilités.

Dépendance matérielle externe. L'intégration d'une caméra 360° permanente n'a pas pu être finalisée, le TechLab ne disposant pas du matériel adapté. L'ANDRA a prêté une caméra Ricoh Theta lors des deux descentes, permettant de constituer le dataset nécessaire à l'entraînement du modèle. L'acquisition du matériel définitif (Insta360 ou équivalent avec driver ROS) reste un prérequis pour l'équipe suivante.

5.6 Transmission inter-années

La transmission du projet d'une équipe à l'autre est le défi structurel de ce projet depuis plusieurs années. Pour éviter de répéter l'expérience du début d'année, plusieurs livrables de continuité ont été produits :

- **Dépôt GitHub** : code, configuration Docker, configuration RViz et `TODO.md` utilisée comme feuille de suivi.
- **Documentation opérationnelle** : plus de 10 guides couvrant le démarrage, le débogage, la visualisation, la navigation, la connexion réseau, entre autres.
- **Image disque NVMe** : transmise sur clé USB avec le robot.
- **Modèles et datasets YOLO** : `best.pt` et `best.engine` du modèle PTZ ; `best.pt` du modèle 360° (YOLOv11s-seg) ; datasets annotés consolidés (451 images PTZ, 344 images 360°).

L'objectif est qu'une équipe 2026-2027 puisse remettre le robot en service en moins d'une demi-journée, sans dépendre d'une connaissance préalable du système.

6. Résultats

Cette section présente un bilan structuré des travaux réalisés tout au long de l'année, en distinguant les objectifs atteints, partiellement atteints et reportés.

6.1 Infrastructure logicielle

La stack ROS2 [10] a été reconstruite à partir des sources de l'équipe 2024-2025, puis réorganisée en trois packages (`image_transfer`, `navigation_utils`, `patrouille_autonome`) et un lanceur central `ros_launcher` (EKF, SLAM/AMCL, drivers Scout/LiDAR/ZED). À l'issue de l'année, **11 nœuds applicatifs** sont opérationnels : capture et contrôle PTZ, séquences photo/vidéo, détection YOLO, rapport de positions sur carte, mission Nav2 et patrouille autonome. Cinq scripts (`setup`, `build`, `launch`, `TensorRT`, `Docker GPU`) et plus de dix guides Markdown, complétés par une image NVMe sauvegardée, visent un lancement et une prise en main rapides par l'équipe suivante.

6.2 Acquisition d'images

La séquence photo (5 prises PTZ [15] par arrêt) couvre désormais l'intégralité de l'arche de la galerie, ce qui a été validé lors de la seconde descente d'avril 2026. La séquence vidéo permet une acquisition continue à 0.06 m/s avec balayage PTZ sinusoïdal. Au total, plus de 500 images ont été capturées via la caméra PTZ lors des deux descentes ; le dataset PTZ annoté consolidé compte **451 images**. Côté 360°, les panoramiques collectées avec la Ricoh Theta prêtée par l'ANDRA ont été converties en un dataset annoté de **344 images** (214 positives, 130 négatives) après prétraitement cubemap.

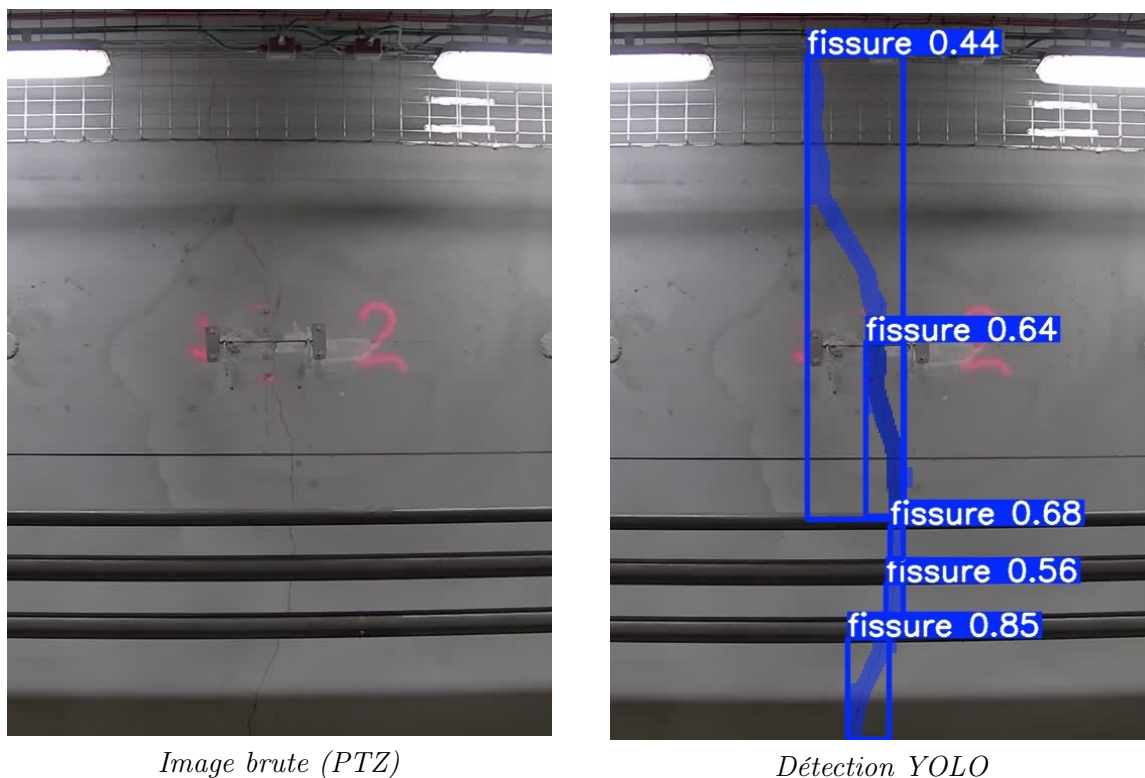


FIGURE 20 – Exemple de détection de fissures sur image PTZ en galerie

6.3 Performances IA

La conversion du modèle PyTorch `best.pt` en moteur TensorRT `best.engine` [17] a permis d'accélérer significativement l'inférence sur la Jetson Orin Nano [16]. Les mesures ont été réalisées en conditions opérationnelles sur la chaîne complète `image_subscriber` (conteneur Docker, images PTZ en 640×640), et non sur l'inférence seule du réseau :

- **PyTorch** : ≈ 125 à 200 ms par image (5 à 8 images/s) ;
- **TensorRT** : ≈ 33 à 40 ms par image (25 à 30 images/s) ;
- **Gain** : facteur $\times 4$ environ (soit une réduction de 75 % du temps de traitement).

Cette accélération a permis de basculer le modèle vers la segmentation d'images dans `image_subscriber` (`task='segment'`), afin d'affiner le contour des fissures au-delà des simples boîtes englobantes. À terme, elle ouvre aussi la voie à une détection quasi temps réel pendant les déplacements du robot.

6.4 Navigation autonome

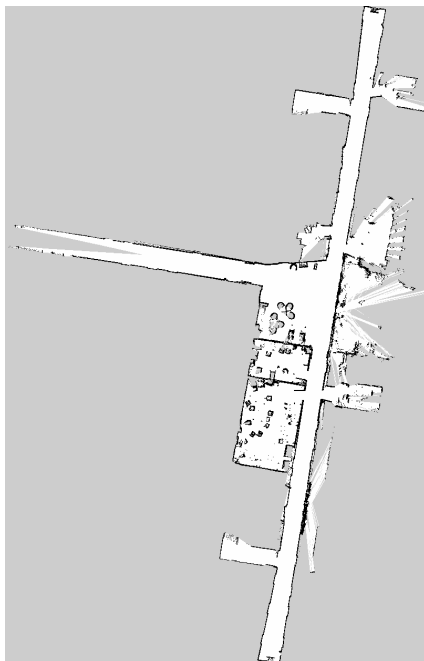
Le robot est capable d'effectuer des missions de navigation autonome sur carte préétablie en mode AMCL [13] + Nav2 [14], à partir d'un fichier de waypoints défini :

- missions de 10 waypoints validées en laboratoire au TechLab ;
- trajectoires définies depuis RViz (2D Nav Goal) et enregistrées en YAML ;
- fusion navigation/acquisition via le package `patrouille_autonome`.

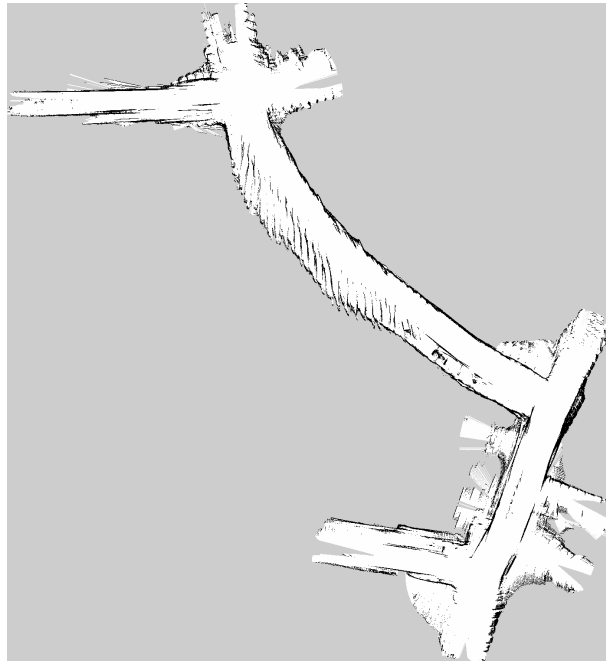
6.5 Cartographie

Les premiers essais de cartographie SLAM [11] dans les tunnels réels ont produit des cartes partielles encourageantes. Cependant, la dérive odométrique rend ces cartes encore difficilement exploitables pour de longs parcours autonomes. En laboratoire, nous avons réussi à fiabiliser cette cartographie en accordant plus d'importance aux scans du LiDAR grâce à la refonte de l'architecture de la tourelle ; les cartes sont désormais exploitables et ont servi de base aux tests de navigation AMCL [13].

Les causes principales de dérive ont été identifiées : voilage de la roue avant gauche (héritage de l'année précédente) et perturbations du LiDAR par les piliers de la tourelle (complètement résolu par la conception des nouvelles platines 3D).



Dérive corrigée — labo TechLab



Avec dérive — tunnel ANDRA

FIGURE 21 – Comparaison de la qualité de cartographie selon l'environnement

6.6 Géolocalisation des fissures

La chaîne `image_subscriber` $\rightarrow$ `/position_detectee` $\rightarrow$ `report_fissures` associe une détection à la pose du robot sur la carte, en s'appuyant sur l'odométrie filtrée [12] et la TF `map` $\leftarrow$ `odom`. À chaque fissure identifiée, les coordonnées sont journalisées en `odom` et en `map`, puis matérialisées par un PNG horodaté (`map_with_point_*.png`). Cette brique clôt le périmètre fonctionnel 2025-2026 : détection, navigation autonome et localisation des fissures sur carte.

6.7 Tableau de synthèse

Objectif	Statut	Commentaire
Infrastructure ROS2	Atteint	Stack complète + docs
Séquence photo PTZ	Atteint	5 prises, arche couverte
Séquence vidéo	Atteint	Fonctionnelle
Inférence TensorRT	Atteint	$\times 4$ sur Jetson
Navigation AMCL + Nav2	Atteint	Validé en labo
Fusion navigation/acquisition	Atteint	Prototype labo validé
Localisation robuste	Atteint	Nouvelle architecture validée
Association image/position	Atteint	Tracé sur carte (PNG) validé
Annotation des datasets	Partiel	PTZ : 451 img. ; 360° : 344 img.
Cartographie galeries	Partiel	À réessayer avec les corrections
Modèle IA 360°	Partiel	YOLOv11s-seg entraîné
Rapports automatiques	Reporté	Création de rapport auto
Amélioration du modèle PTZ	Reporté	Ré-entraînement PTZ à faire
IA tous types de murs	Reporté	Actuellement limité au GER

TABLE 3 – Bilan des objectifs de l'année 2025-2026

7. Conclusion et Ouvertures

7.1 Conclusion

Ce projet nous a confrontés, dès les premières semaines, à une réalité courante en ingénierie : la partie la plus difficile n'est pas toujours la conception, mais la reprise d'un système existant dont les fondations ont disparu. La perte de l'image système du robot a imposé une réinstallation complète et une compréhension approfondie de chaque composant avant de pouvoir avancer. Malgré ce point de départ contraint, l'équipe a franchi trois étapes clés : une acquisition fiable et automatisée des images de galerie, un premier déplacement autonome sur carte préétablie, et l'association de chaque fissure détectée à sa position sur la carte. Le pipeline de détection PTZ est accéléré grâce à TensorRT [17], un premier modèle de segmentation 360° a été entraîné en laboratoire, le robot effectue des missions autonomes avec Nav2 [14], la fusion navigation/acquisition est prototypée, et la chaîne image/position est opérationnelle.

7.2 Ouvertures

Les pistes d'amélioration sont organisées par priorité pour guider l'équipe 2026-2027. La boucle détection + navigation + géolocalisation des fissures sur carte est en place ; les éléments ci-dessous visent surtout la montée en maturité terrain et les extensions (360°, type de mur, rapports historiques).

Plusieurs limites subsistent. La dérive odométrique dégrade la cartographie : nous ne disposons pas encore d'une carte fiable pour y mener des missions autonomes reproductibles, malgré des validations convaincantes au TechLab. Côté détection, le modèle YOLO PTZ en production reste limité au revêtement de type GER ; le modèle 360° entraîné n'est pas encore branché au robot. Enfin, l'absence de caméra 360° permanente au TechLab empêche sa validation en conditions de patrouille autonome.

Court terme — reprise par l'équipe suivante.

- **Séquences en mode patrouille** : affiner `sequence_photo` et `sequence_video` lorsqu'elles sont orchestrées par `patrouille_autonome` : adapter les timings PTZ ainsi que les déclenchements de capture au trajet et au rythme de la patrouille et vérifier que l'arche reste couverte sur l'ensemble du parcours.
- **Modèle 360°** : intégrer le `best.pt` YOLOv11s-seg au robot, compléter les annotations restantes via pseudo-labeling et améliorer le rappel.
- **Ré-entraînement PTZ** : lancer un entraînement sur le dataset consolidé.
- **Enrichir report_fissures** : ajouter historique et comparaison inter-rondes au tracé carte pour générer des rapports détaillés de chaque fissure détectée.

Moyen terme — fiabilité du système.

- **Cartographie exploitable** : réduire la dérive odométrique (roue avant gauche voilée) et explorer l'usage des étiquettes présentes dans les tunnels ANDRA comme ancres de recalibrage. (idée de l'équipe 2024-2025)
- **Intégration caméra 360° permanente** : acquérir un modèle compatible ROS (Insta360, Ricoh Theta ou équivalent) et l'intégrer au robot comme capteur permanent, remplaçant définitivement la caméra PTZ pour la détection de fissures.
- **Cartes des galeries** : produire des cartes SLAM [11]/AMCL [13] exploitables des galeries visitées pour permettre une navigation autonome reproductible. (il existe déjà des cartes très fiables des tunnels créées par les ingénieurs du TechLab)

Long terme — vision ANDRA.

- **Analyse continue** : coupler la caméra 360° avec un modèle adapté pour une détection de fissures en temps réel pendant les rondes, sans arrêt du robot.
- **Rapports automatiques complets** : généraliser la génération de rapports d'état des fissures avec suivi temporel sur l'ensemble des galeries.
- **Extension de l'IA** : entraîner le modèle sur tous les types de revêtement de mur des galeries ANDRA, actuellement limité au type GER.
- **Autonomie complète** : planification automatique des rondes d'inspection et synchronisation directe des rapports vers une base de données centralisée.

Références

- [1] AGENCE NATIONALE POUR LA GESTION DES DÉCHETS RADIOACTIFS (ANDRA). *Cigéo — Dossier du maître d’ouvrage*. Accessed : 2025-05-18. 2024. URL : <https://www.andra.fr/cigeo>.
- [2] UNIVERSITY. *crack Dataset*. Open Source Dataset. visited on 2024-01-23. Déc. 2022. URL : <https://universe.roboflow.com/university-bswxt/crack-bphdr>.
- [3] Glenn JOCHER, Ayush CHAURASIA et Jing QIU. *Ultralytics YOLO*. Accessed : 2025-05-18. 2023. URL : <https://github.com/ultralytics/ultralytics>.
- [4] AGILEX ROBOTICS. *Scout Mini, User Manual and Platform Documentation*. Accessed : 2025-05-18. 2023. URL : <https://global.agilex.ai/products/scout-mini>.
- [5] Agilex ROBOTICS. *scout_ros2*. Accessed : 2025-05-30. 2023. URL : https://github.com/agilexrobotics/scout_ros2.
- [6] YDLIDAR. *TG Series 2D LiDAR — Product Documentation*. Accessed : 2025-05-18. 2023. URL : <https://www.ydlidar.com/>.
- [7] YDLIDAR. *ydlidar_ros2_driver*. Accessed : 2025-05-30. 2023. URL : https://github.com/YDLIDAR/ydlidar_ros2_driver.
- [8] STEREO LABS. *ZED 2i — Stereo Camera Technical Specifications*. Accessed : 2025-05-18. 2023. URL : <https://www.stereolabs.com/zed-2i>.
- [9] STEREO LABS. *zed_ros2_wrapper*. Accessed : 2025-05-30. 2023. URL : <https://github.com/stereolabs/zed-ros2-wrapper>.
- [10] OPEN ROBOTICS. *ROS 2 Humble — Documentation officielle*. Consultée en 2025-2026. 2022. URL : <https://docs.ros.org/en/humble/>.
- [11] Steve MACENSKI et Ivona JAMBRECIC. « SLAM Toolbox : SLAM for the dynamic world ». In : *Journal of Open Source Software* 6.61 (2021), p. 2783. DOI : 10.21105/joss.02783. URL : https://github.com/SteveMacenski/slam_toolbox.
- [12] Tom MOORE et Daniel STOUCH. *A Generalized Extended Kalman Filter Implementation for the Robot Operating System*. Accessed : 2025-05-18. 2016. URL : https://github.com/cra-ros-pkg/robot_localization.
- [13] NAVIGATION2 PROJECT. *Configuring AMCL*. <https://docs.nav2.org/configuration/packages/configuring-amcl.html>. Accessed : May 30, 2025. 2024.
- [14] Steve MACENSKI et al. « The Marathon 2 : A Navigation System ». In : *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, p. 2718-2725. DOI : 10.1109/IROS45743.2020.9341207. URL : <https://navigation.ros.org/>.
- [15] MARSHALL ELECTRONICS. *CV-605 HD60 IP PTZ Camera — User Manual and VISCA over IP Protocol Reference*. Accessed : 2025-05-18. Marshall Electronics. 2022. URL : <https://www.marshall-usa.com/cameras/CV-605.php>.
- [16] NVIDIA CORPORATION. *Jetson Orin Nano Developer Kit — Jetson Linux Developer Guide*. Accessed : 2025-05-18. 2023. URL : <https://developer.nvidia.com/embedded/jetson-orin-nano-devkit>.
- [17] NVIDIA CORPORATION. *TensorRT Developer Guide*. Accessed : 2025-05-18. 2024. URL : <https://developer.nvidia.com/tensorrt>.